

Taller Implementación de tácticas de desempeño: Introducir Concurrencia y Mantener Múltiples Copias de Datos (caché)

1. Contexto del problema

RECAUDO-T (el mismo servicio de consulta de saldo del taller de disponibilidad) añadió una funcionalidad muy utilizada: el resumen mensual de viajes, que cada usuario consulta en la aplicación móvil para ver cuánto ha gastado durante el mes. Calcular ese resumen es costoso: hay que revisar todos los viajes del mes y sumarlos. Al liberar la función al inicio del mes (cuando más gente consulta cuánto gastó), el servicio se volvió muy lento: las respuestas tardaban varios segundos y muchas solicitudes ni siquiera alcanzaban a completarse. Le piden a su equipo que mejore el desempeño del servicio bajo esa carga, sin cambiar lo que hace el cálculo, sino cómo se ejecuta.

Escenario de calidad de desempeño

Parte del escenario	Valor
Fuente del estímulo	Usuarios (muchas solicitudes simultáneas desde la aplicación móvil)
Estímulo	Ráfaga de solicitudes concurrentes al resumen mensual (por ejemplo, 100 solicitudes en 10 segundos)
Respuesta esperada	El sistema sigue respondiendo en un tiempo aceptable y no descarta solicitudes
Medida de la respuesta	Latencia p95 dentro de un límite razonable (por ejemplo, 300 ms); throughput sostenido; sin solicitudes descartadas

2. Marco conceptual

El desempeño es la capacidad del sistema para responder a tiempo a los eventos que le llegan (interrupciones, mensajes, solicitudes de usuarios). Los eventos pueden llegar de tres formas: periódicos (a intervalos regulares), estocásticos (según una distribución de probabilidad) o esporádicos (sin un patrón definido). Las solicitudes de este taller son esporádicas: llegan en ráfagas impredecibles, como ocurre al inicio de un mes real.

Para medir el desempeño se usan cuatro medidas: latencia (tiempo entre que llega la solicitud y que el sistema responde), throughput (número de solicitudes que el sistema procesa por unidad de tiempo), jitter (qué tanto varía el tiempo de respuesta entre solicitudes consecutivas) y eventos no procesados (solicitudes que el sistema no alcanza a atender por estar demasiado ocupado).

Este taller trabaja dos tácticas de la rama Gestionar los recursos:

- Introducir concurrencia: procesar varias solicitudes al mismo tiempo en lugar de una por una, para que el tiempo total no crezca proporcionalmente al número de solicitudes.
- Mantener Múltiples Copias de Datos (caché): guardar el resultado de un cálculo ya realizado, de modo que una solicitud repetida se responda de inmediato en lugar de recalcularlo.

3. Cómo funciona el sistema

El servidor debe calcular el resumen mensual de un usuario sumando sus viajes registrados. Ese cálculo, a propósito, requiere un tiempo fijo apreciable (por ejemplo, 200 ms), lo que simula el costo real de revisar muchos registros.

Si el servidor atiende las solicitudes una por una, en el orden en que llegan, el tiempo de espera de cada solicitud crece con el número de solicitudes que la anteceden: la solicitud número 50 debe esperar a que se completen las 49 anteriores antes de empezar siquiera a procesarse. Por eso, el servidor debe ser capaz de calcular varios resúmenes simultáneamente, utilizando varios hilos o procesos de trabajo en lugar de uno solo. Con esto, mientras uno de los trabajadores calcula el resumen de un usuario, otro puede calcular el de otro en simultáneo.

Además, antes de realizar el cálculo, el servidor debe verificar si ya calculó recientemente el resumen de ese usuario. Si lo tiene guardado en caché, debe devolverlo de inmediato, sin repetir el cálculo. Si no lo tiene, debe calcularlo, guardarlo y luego responder. La segunda vez que alguien pida el mismo resumen, la respuesta debe ser casi instantánea.

Estas dos mejoras actúan en momentos distintos: la concurrencia ayuda cuando muchas solicitudes distintas llegan a la vez; el caché ayuda cuando una misma solicitud se repite. Un sistema real necesita ambas, porque no siempre son las mismas personas las que preguntan, pero muchas sí preguntan por segunda vez en la misma sesión.

4. Arquitectura y contrato de cada componente

Aclaración: no hay usuarios reales ni hardware especial. El Cliente es un script que dispara solicitudes automáticamente en poco tiempo, simulando una ráfaga real de usuarios que consultan su resumen al inicio de mes; no es una interfaz gráfica.

```
CLIENTE (script)
| GET /resumen/{id}
▼
SERVIDOR
| revisa el caché
```

└─ HIT —► responde de inmediato (desde_cache=true)
└─ MISS —► encola en el pool de trabajadores —► calcula —► guarda en caché —► responde

GET /config → consulta cuántos trabajadores están activos y cuántas entradas tiene el caché

SERVIDOR

Único proceso:

Ruta	Uso
GET /resumen/{idTarjeta}?viajes=N	Calcula (o recupera de caché) el resumen de los últimos N viajes. Respuesta 200: {"id_tarjeta":"1234","total_viajes":40,"gasto_total":152000,"desde_cache":true}
GET /config	Devuelve cuántos trabajadores están activos y cuántas entradas tiene el caché en ese momento (le sirve para verificar en vivo que la concurrencia y el caché funcionan)

CLIENTE

Script que, durante DURACION segundos, a ritmo alto (ráfaga), dispara GET /resumen/{idTarjeta} reutilizando un conjunto pequeño de tarjetas (sugerido: 10) para que se repitan las consultas y el caché tenga oportunidad de responder. Por cada intento, registra en un CSV: timestamp_envio, timestamp_respuesta, éxito (0/1), desde_cache (0/1), id_tarjeta.

5. Requisitos a implementar

R1 Introducir Concurrencia

- El servidor atiende varias solicitudes simultáneamente mediante un conjunto de trabajadores (hilos, procesos o tareas asíncronas). El número de trabajadores (W) debe ser configurable.
- Cuando llegan más solicitudes que trabajadores disponibles, las adicionales esperan en una cola; no se pierden mientras la cola no exceda el límite del numeral de Restricciones.

R2 Mantener Múltiples Copias de Datos (caché)

- Antes de calcular, el servidor verifica si ya tiene guardado el resultado para esa tarjeta (y para el número de viajes consultado).
- Si lo tiene, responda con el valor guardado y desde_cache=true, sin ejecutar el cálculo.
- Si no lo tiene, lo calcula, lo guarda y responde con desde_cache=false.

Restricciones

1. El cálculo costoso debe implementarse de verdad, con una duración fija y comparable entre corridas (por ejemplo, siempre entre 150 y 250 ms), para que los experimentos sean comparables. No delegue la concurrencia completa a un framework que ya la resuelva sin que usted configure el número de trabajadores.
2. El caché es en memoria (no requiere base de datos externa) y no necesita expirar durante los experimentos de este taller.

6. Cómo medir: latencia, throughput, jitter y eventos no procesados

Con el CSV del cliente tiene suficiente evidencia para las cuatro medidas de la teoría:

Métrica	Cómo se calcula
Latencia (promedio y p95)	timestamp_respuesta menos timestamp_envio, por solicitud exitosa
Throughput	solicitudes exitosas dividido entre la duración total del experimento
Jitter	diferencia, en valor absoluto, entre la latencia de una solicitud y la de la solicitud inmediatamente anterior, promediada
Eventos no procesados	solicitudes que no obtuvieron respuesta dentro de un tiempo límite razonable (sugerido: 3 s), contadas como éxito=0

Calcule, además, el porcentaje de respuestas servidas desde caché: filas con desde_cache=1 dividido entre el total. Adjunte siempre el CSV de donde proviene cada número; un número sin evidencia no cuenta como medición.

7. Protocolo experimental

El cliente dispara una ráfaga de al menos 100 solicitudes en 15 a 20 segundos, reutilizando 10 tarjetas.

E0 Línea base. Un solo trabajador (W=1), sin caché. Muestra el problema tal como llegó: en ráfaga, la latencia y el jitter aumentan y pueden aparecer solicitudes no procesadas.

E1 Con concurrencia. Mismo experimento, ahora con varios trabajadores (sugerido: W=8), sin caché todavía. Compare el throughput y la latencia frente a E0: el throughput debería subir bastante. La latencia de cada solicitud individual no necesariamente disminuye (el cálculo sigue tardando lo mismo), pero se atienden muchas más solicitudes en el mismo tiempo.

E2 Con concurrencia y caché. Active también la caché (R2). Como el cliente reutiliza pocas tarjetas, la mayoría de las solicitudes deberían encontrar el resultado ya calculado. Mida qué porcentaje de respuestas provino de la caché y compare su latencia promedio con la de las que sí requirieron cálculo. Ejecútelo dos veces para confirmar los valores.

8. Entregables

1. Código fuente con instrucciones para levantarlo con un solo comando.
2. CSV del cliente de al menos una corrida de cada experimento (E0, E1, E2).
3. Tabla con las cuatro métricas y el porcentaje de respuestas desde caché, para los tres experimentos.
4. Media página respondiendo las preguntas de análisis del numeral 9.

Verificación sugerida antes de entregar el taller: el servidor responde; hay una diferencia medible en el throughput entre $W=1$ y $W=8$; el campo `desde_cache` distingue entre respuestas calculadas y servidas desde el caché.

9. Preguntas de análisis

1. Compare el throughput de E0 y E1. ¿Por qué aumentar el número de trabajadores mejora el throughput aunque cada cálculo individual siga tardando lo mismo?
2. En E1, ¿bajó la latencia promedio de cada solicitud individual, o solo mejoró cuántas solicitudes se atendieron en total? Explique la diferencia entre esos dos efectos.
3. En E2, ¿qué porcentaje de solicitudes provino del caché? ¿Qué pasaría con ese porcentaje si el cliente usara 1000 tarjetas distintas en vez de 10? ¿Siguiendo siendo útil el caché en ese caso?
4. Si el servidor aumentara indefinidamente el número de trabajadores para atender más solicitudes a la vez, ¿qué límite físico o técnico lo detendría con el tiempo? (Relacione su respuesta con la táctica “Aumentar los Recursos” del árbol.)
5. El caché evita repetir el cálculo, pero ¿qué pasa si el usuario realiza un nuevo viaje después de que su resumen quedó guardado en caché? ¿Qué problema aparece y qué tendría que agregar para resolverlo?